UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Chapter 1: Introduction to Object-Oriented Programming & Java (Part 1)

SECJ2154: Object-Oriented Programming
Faculty of Computing (FC), UTM

Note: This slide is based on Dr. Norazah Yusof's slide
Updated by Shahida Sulaiman, 2025
Other sources are cited accordingly
Reminder: Do not upload this slide to external platforms

Innovating Solutions

UTM JOHOR BAHRU

Contents

- **Introduction:**
  - Procedural Programming vs. Object-Oriented Programming (OOP)
- **Object-Oriented (OO) Definition:**
  - Vocabulary of OOP
- **Object-Oriented Concept:**
  - 7 Principles of Object Orientation
  - Benefits of OOP

Procedural Programming

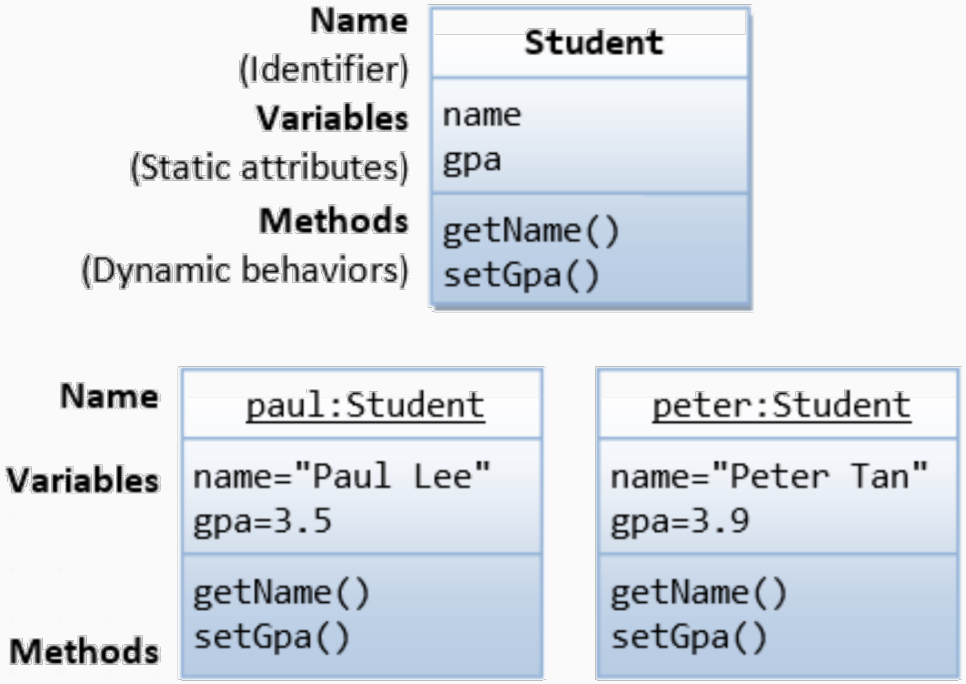
INTRODUCTION

- Traditional programming languages were procedural e.g.
  - C, Pascal, BASIC, Ada and COBOL
- Programming in procedural languages involves choosing **data structures** (appropriate ways to store data), designing algorithms, and translating algorithms into code.
- In procedural programming, **data and operations** on the data are **separated**.
- This methodology requires sending data to procedure or functions.

Object-Oriented Programming (OOP)

INTRODUCTION

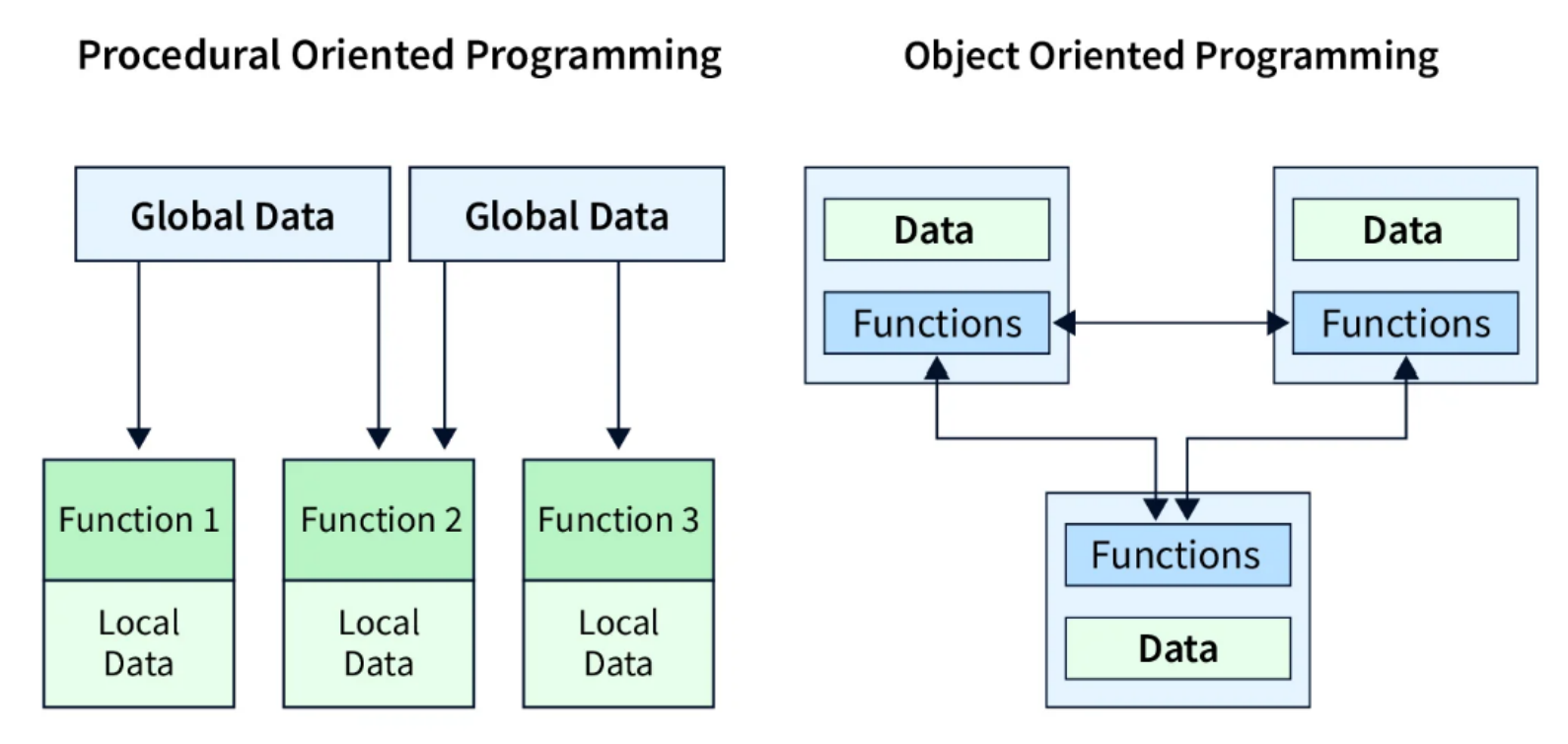
- OOP is centered on creating **objects** rather than procedures or functions.
- Objects are a combining of **data** and **procedures** that manipulate that data.
- Data in an object are known as ***attributes***.
- Procedures/functions in an object are known as ***methods***.



Two instances - paul and peter - of the class Student

Source:https://www3.ntu.edu.sg/home/ehchua/programming/java/J3a_OOPBasics.html

Procedural vs. OOP

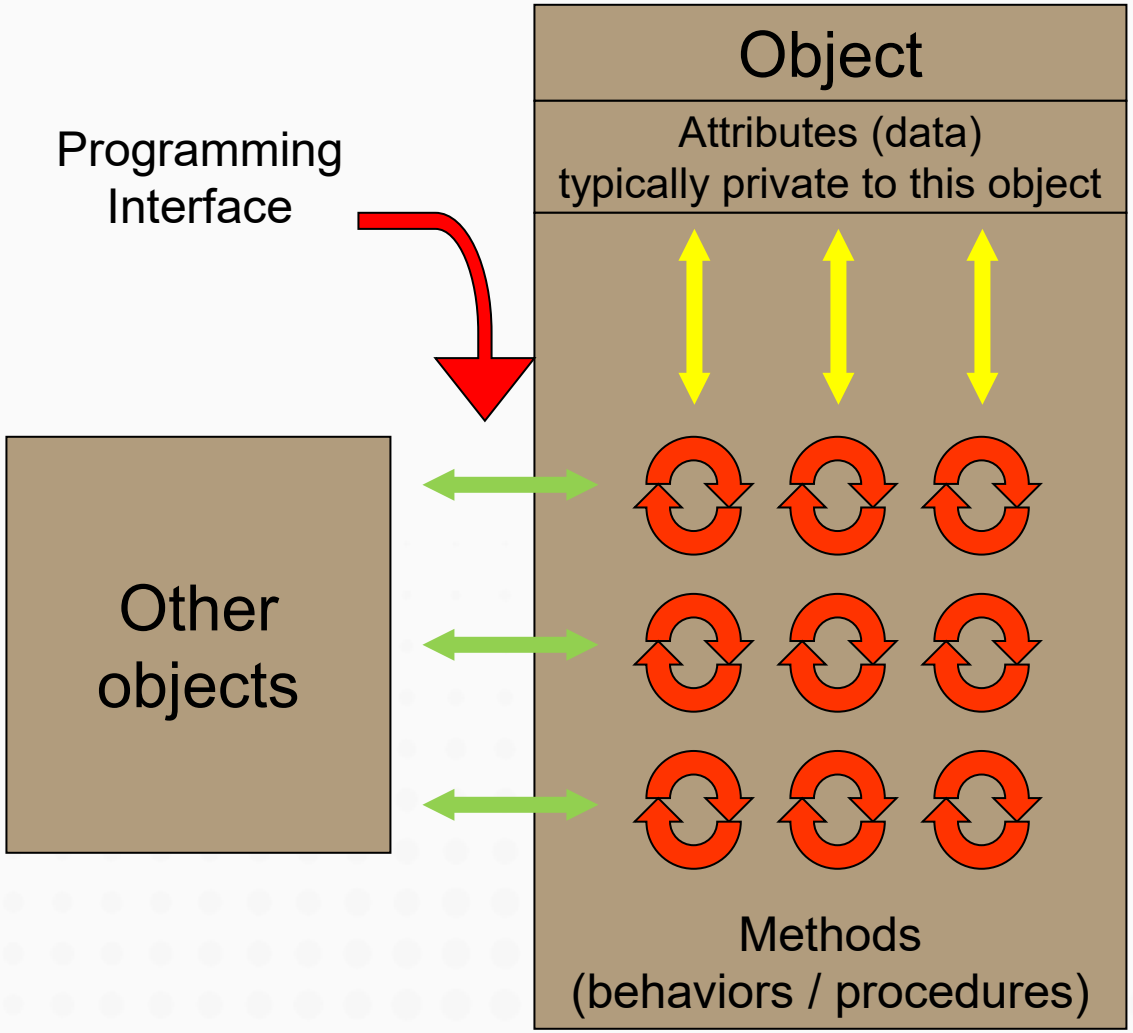


Source: https://www.scaler.com/topics/difference-between-oop-and-pop/

OOP & Programming Interface

- OOP combines data and behavior (or method) via **encapsulation**.
- **Data hiding** is the ability of an object to hide data from other objects in the program.
- Only **methods in an object** should be able to **directly manipulate its attributes**.
- **Other objects** are allowed to manipulate an object's attributes via the object's methods.
- This indirect access is known as a **programming interface**.

OOP & Programming Interface



OOP Languages

- **Pure** OO Languages: e.g. Smalltalk, Eiffel, Ruby
  - Have strict features e.g. treat everything inside the source code as objects, do not support primitive data types (e.g. int, char, float, bool)
- **Hybrid** OO Languages: e.g. C++, Objective-C, Phyton, Java
  - Supports both procedural and OOP paradigms
- Thus, **Java** is an OOP Language **but not pure OOP** as it supports primitive data, which are not objects

(Source: https://www.naukri.com/code360/library/is-java-an-object-oriented-language)

Vocabulary of OOP: Class

OO DEFINITION

- A class is the **template or blueprint** from which objects are made.
- All **codes** written in Java are **inside a class**.
- The standard Java library supplies several thousand classes.
- Create **classes in Java** to describe the objects of the problem domains of applications and to adapt the classes that are supplied by the standard library for specific purposes.
- A class **encapsulates the attributes and actions** that characterizes a certain type of object

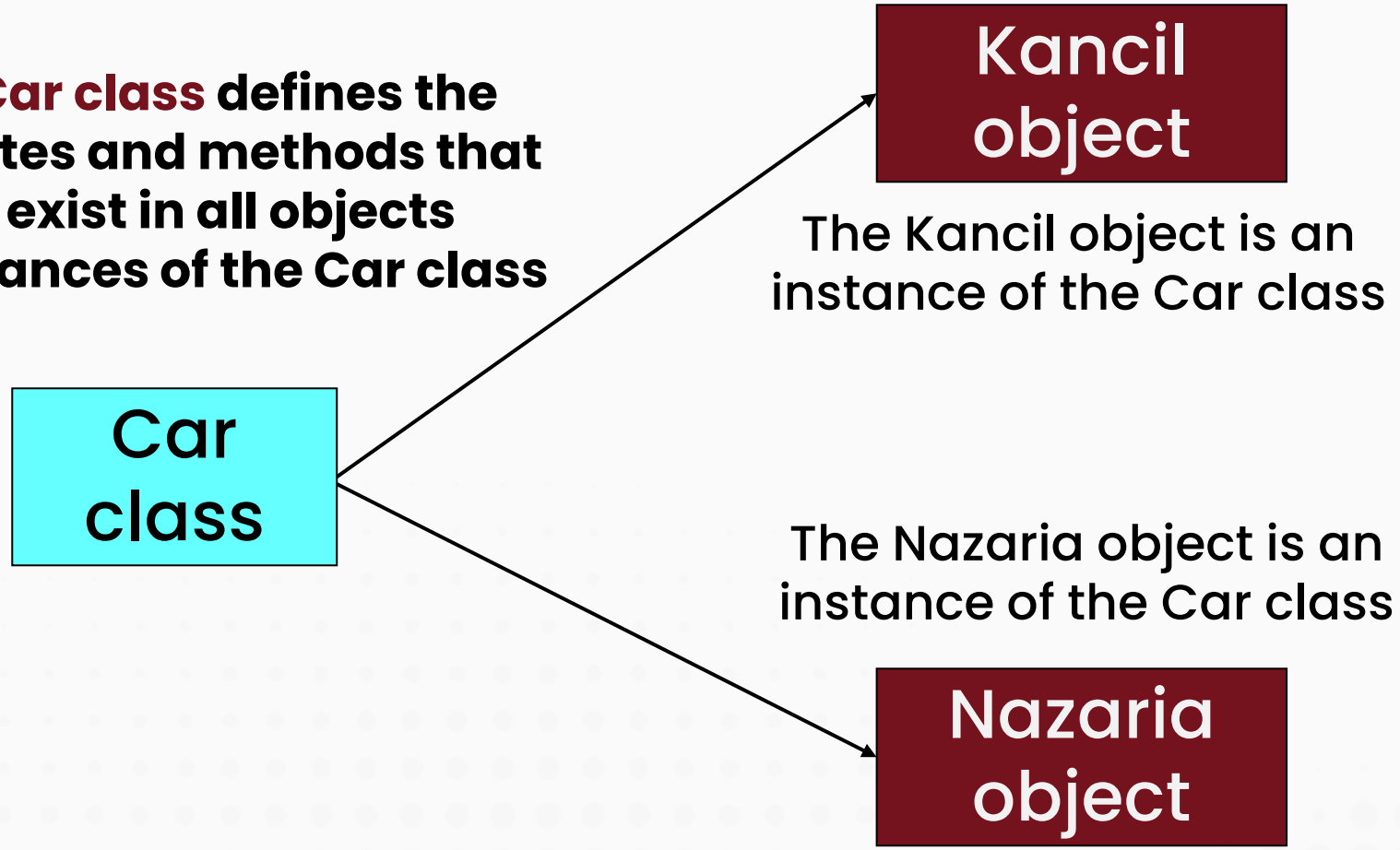
Vocabulary of OOP: Object

OO DEFINITION

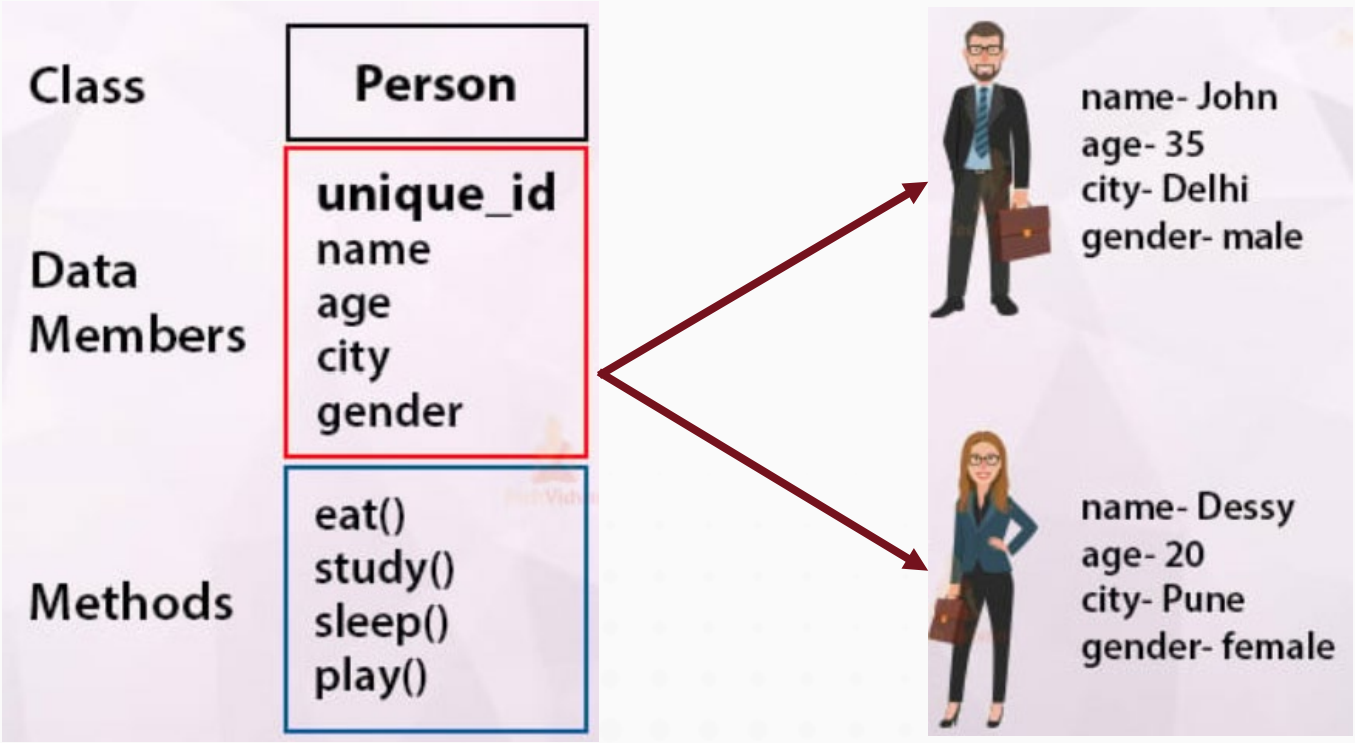
- Classes can be used to **instantiate** as many objects as are needed.
- Each object that is created from a class is called an **instance** of the class.
- A **program** is simply a **collection of objects** that interact with each other to accomplish a goal.

Class and Object: Example 1

The Car class defines the attributes and methods that will exist in all objects i.e. instances of the Car class



Class and Object: Example 2



Source: https://techvidvan.com/tutorials/java-class/

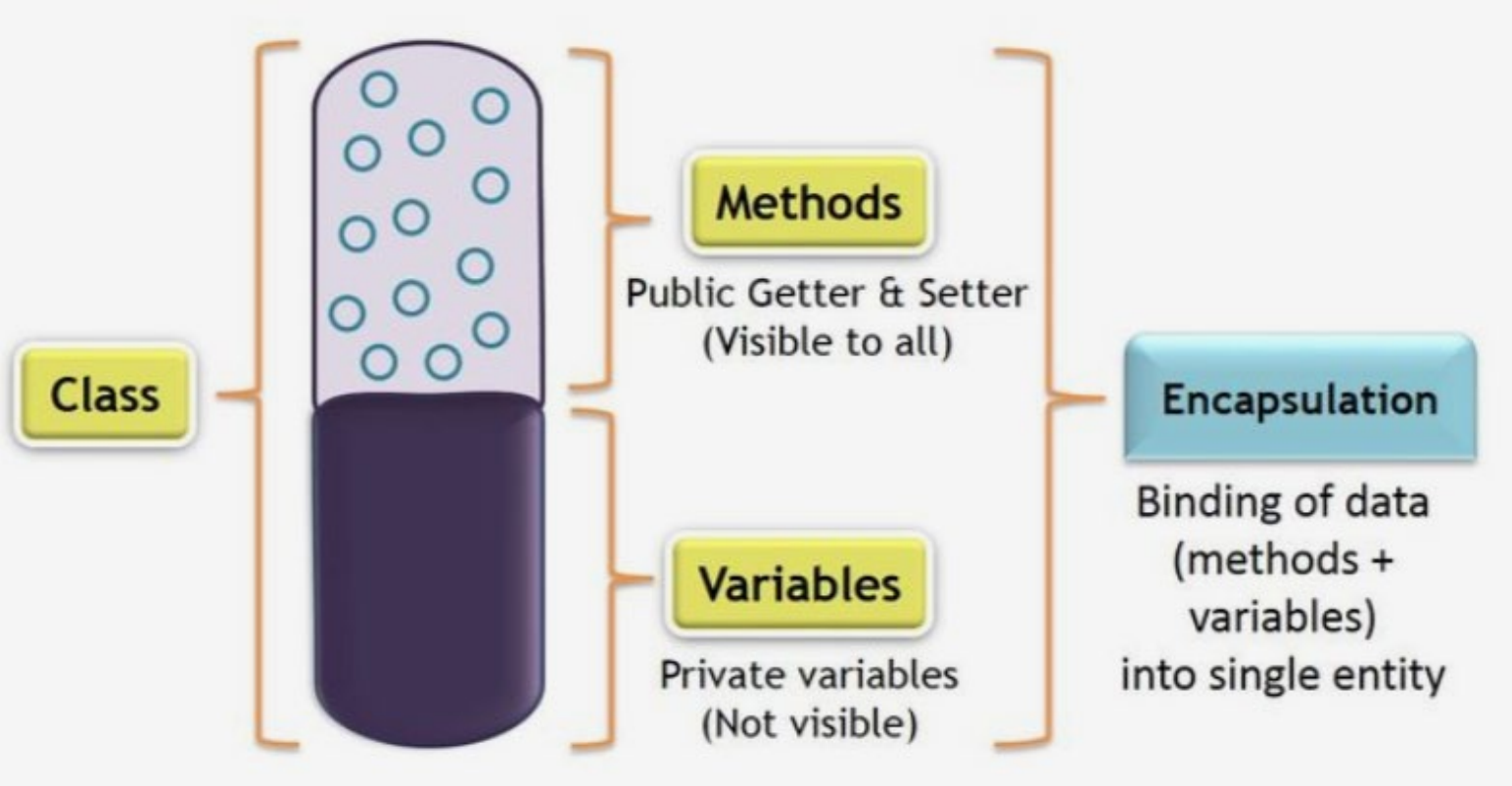
Vocabulary of OOP: Encapsulation

OO DEFINITION

- Encapsulation is a key concept in working with objects
- Combine data and behavior in one package, **hide the implementation of the data** from the user of the object
- The data in an object are called by its instance fields, and the procedures that operate on the data are called by its methods.
- Methods do not directly access instance fields in a class other than their own, programs should **interact with object data** through the object's methods.
- Maximize reusability, reduce data dependency, and minimize debugging time by encapsulating object data

OO DEFINITION

Vocabulary of OOP: Encapsulation



Source: https://corejavasea.blogspot.com/2020/05/encapsulation.html

Vocabulary of OOP: Data Hiding

OO DEFINITION

- Data hiding is a **subset of encapsulation** that refers to the practice of restricting access to certain parts of an object's data by making it private (or protected)
- Data hiding is **important** for several reasons:
  - Protects the attributes from accidental corruption by outside objects
  - Hides the details of how an object works, so the programmer can concentrate on using it
  - Allows the maintainer of the object to have the ability to modify the internal functioning of the object without “breaking” someone else's code

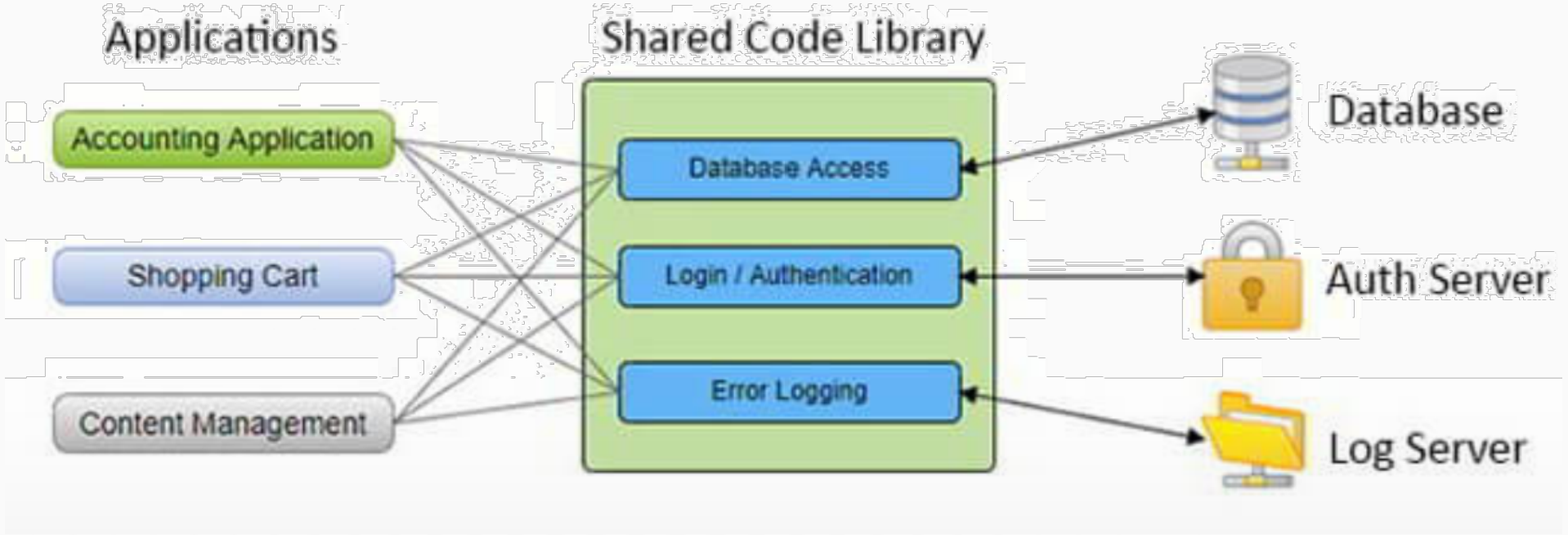
Vocabulary of OOP: Code Reusability

OO DEFINITION

- OOP encourages **component** reusability
- A component is a **software object** contains data and methods that represent a **specific concept or service**.
- Components typically are not stand-alone programs.
- Components can be used by programs that need the **component's service**.
- Reuse of code promotes the rapid development of larger software projects.

Vocabulary of OOP: Code Reusability-Example

OO DEFINITION



Source: https://dev.to/pluralsight/reusable-code-the-good-the-bad-and-the-ugly-3do6

Vocabulary of OOP: Inheritance

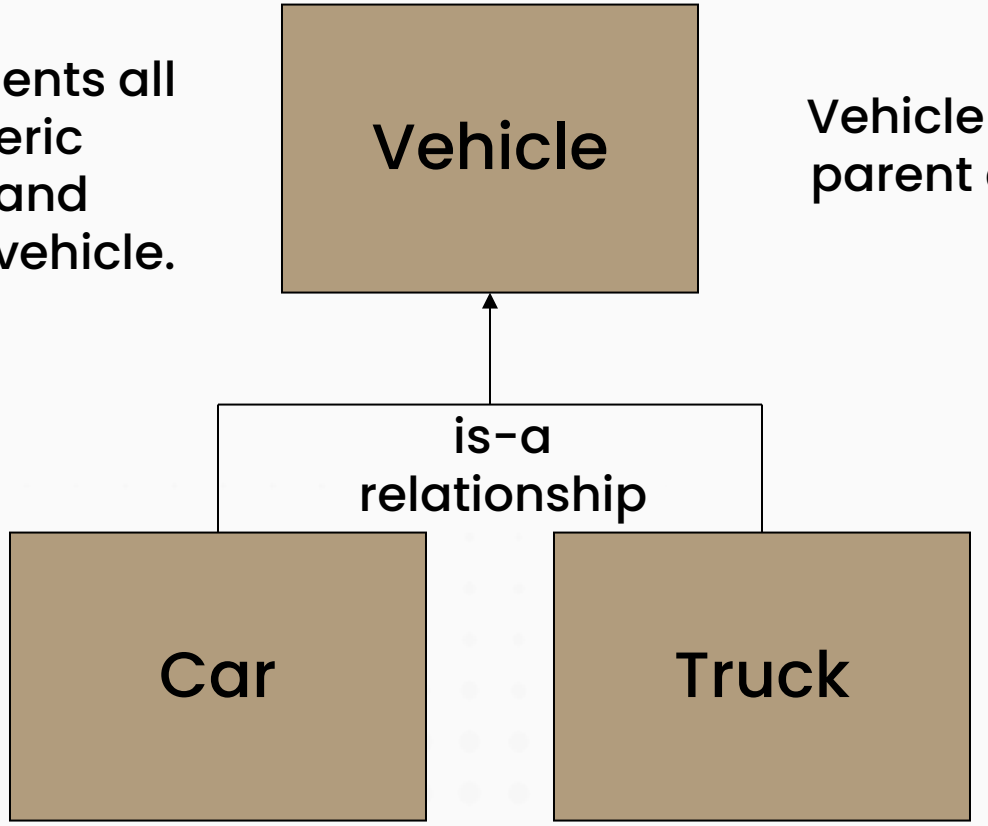
OO DEFINITION

- Inheritance is the ability of one class to **extend the capabilities** of another:
  - It allows **code** defined in one class to be **reused** in other classes
- Consider the class Car. A Car is a **specialized form** of the Vehicle class.
  - The Vehicle class is the **base or parent class** of the Car class
  - The Car class is the **derived or child class** of the Vehicle class

Vocabulary of OOP: Inheritance-Example

OO DEFINITION

Vehicle represents all of the generic attributes and methods of a vehicle.



Vehicle is the parent class.

Car and Truck are child classes of Vehicle.

Car and Truck are Specialized versions of a Vehicle.

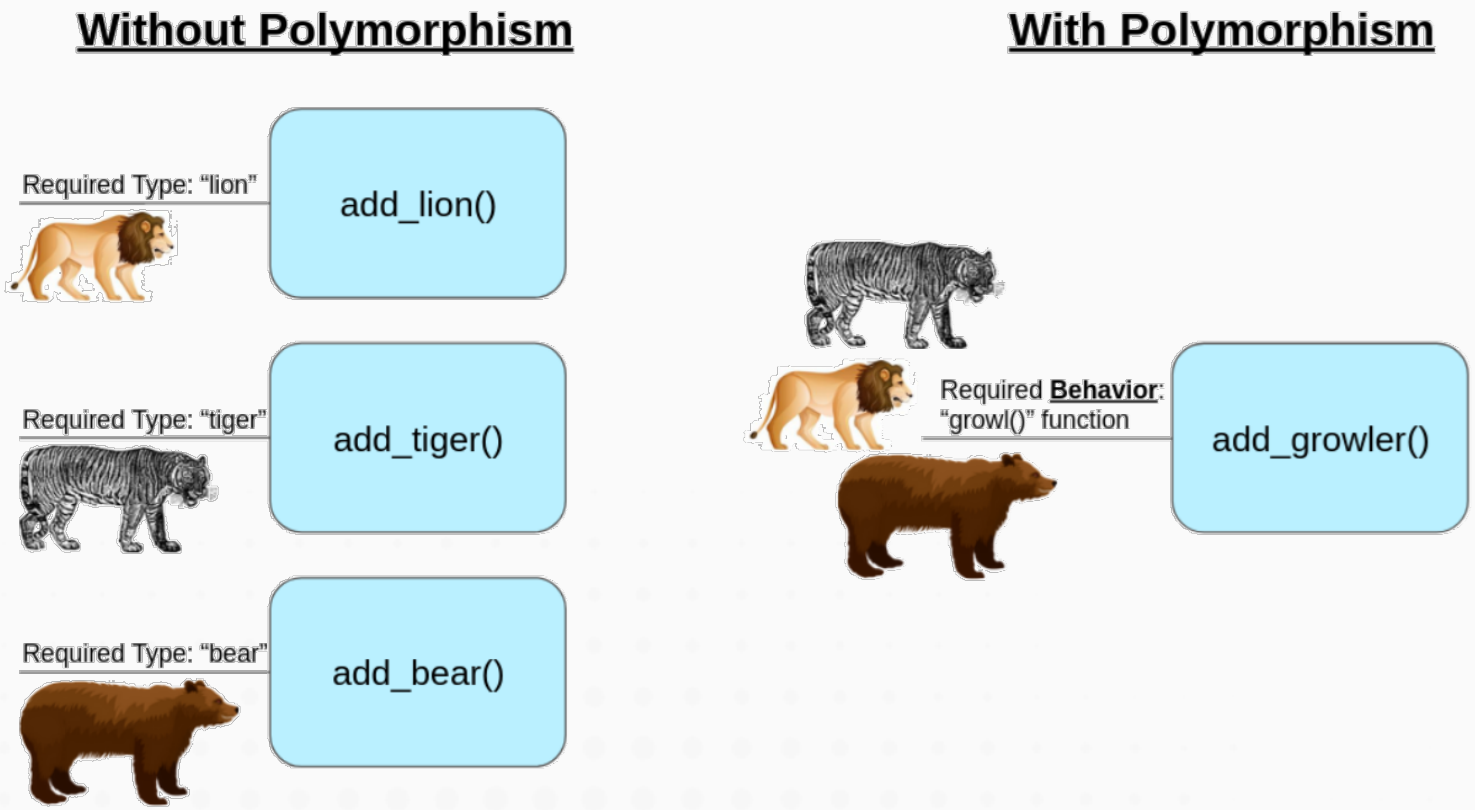
Vocabulary of OOP: Polymorphism

OO DEFINITION

- Polymorphism means "many forms"
  - A **reference variable** is always of a single, unchangeable type, but it can refer to a subtype object
  - A single object can be referred to by **reference variables of many different types**, as long as they are the same type or a supertype of the object
  - The reference variable type (not the object type) determines **which methods can be called**
- Polymorphism allows extensibility

Vocabulary of OOP: Polymorphism-Example

OO DEFINITION



Source: https://oswalt.dev/2021/06/polymorphism-in-rust/

Exercise

SELF-TEST

1. State the differences between **procedural** programming and **OOP**.
2. What is an **Object** and what is a **Class**? What is the difference between them?
3. What is an **Attribute**?
4. What is a **Method**?
5. What is **encapsulation**? How it relates to **information hiding**?
6. What is **inheritance**? How it relates to **polymorphism**?

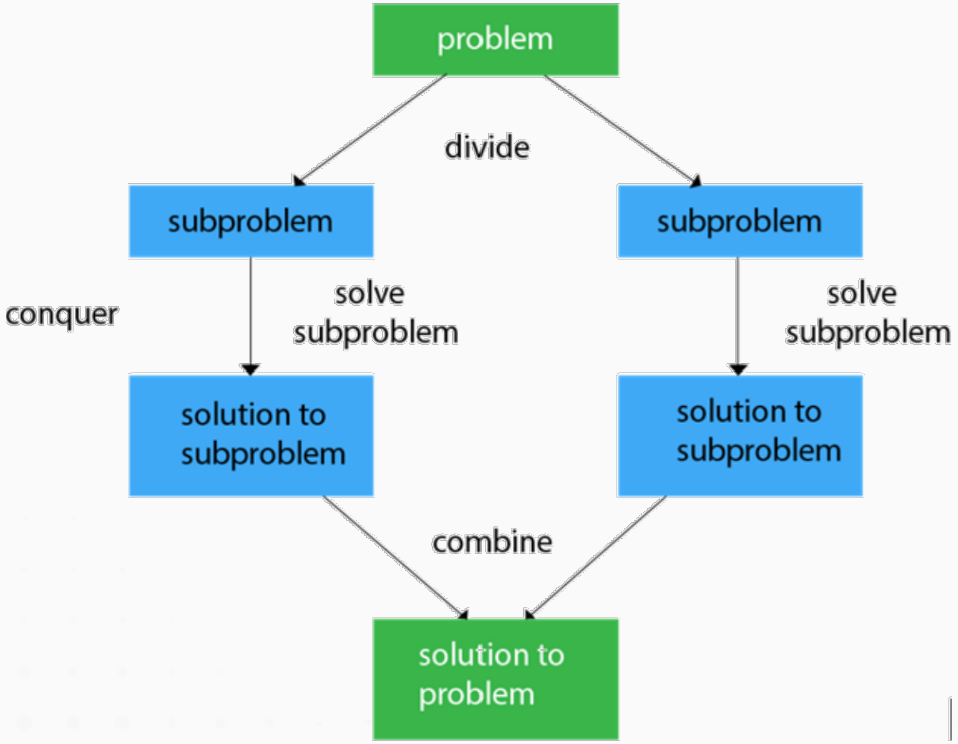
Object Orientation Principles

1. Divide-and-Conquer
2. Encapsulation and Modularity
3. Public Interface
4. Information Hiding
5. Generality
6. Extensibility
7. Abstraction

Principle #1: Divide-and-Conquer

OO CONCEPT

- The **first step** in designing a program is to **divide** the overall program into a number of objects
- The objects will interact with each other to solve the problem
- Problem-solving: Break problems (programs) into **small, manageable tasks**

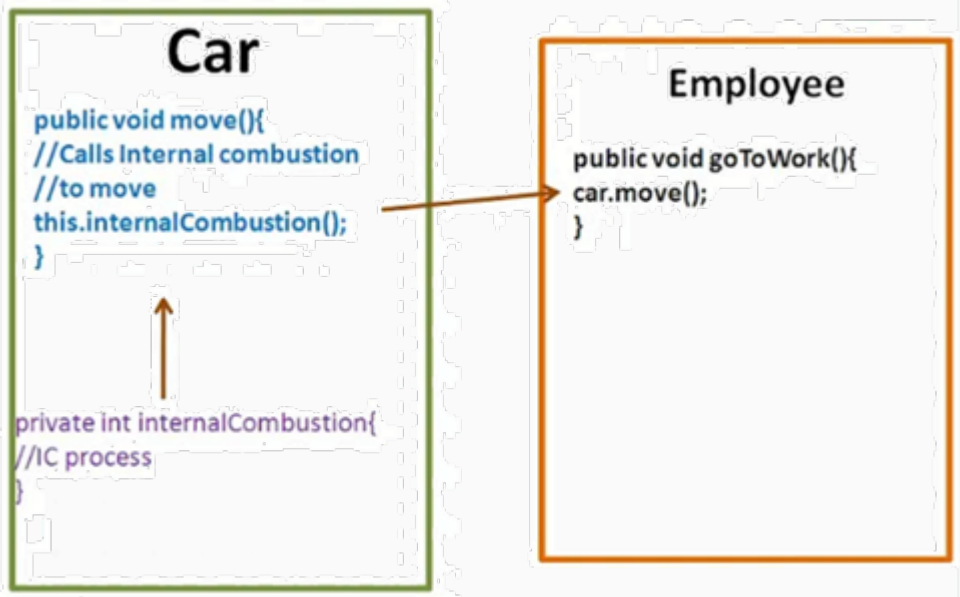


Source: https://www.tpointtech.com/divide-and-conquer-introduction

Principle #2: Encapsulation & Modularity

OO CONCEPT

- The next step in designing a program is to decide for **each object**, what **attribute** it has and what **actions** it will take.
- The goal is that each object is a **self-contained module** with a clear responsibility and the attributes and actions necessary to carry out its role.
- Problem-solving: Each object knows how to **solve its task** and **has the information** it needs.

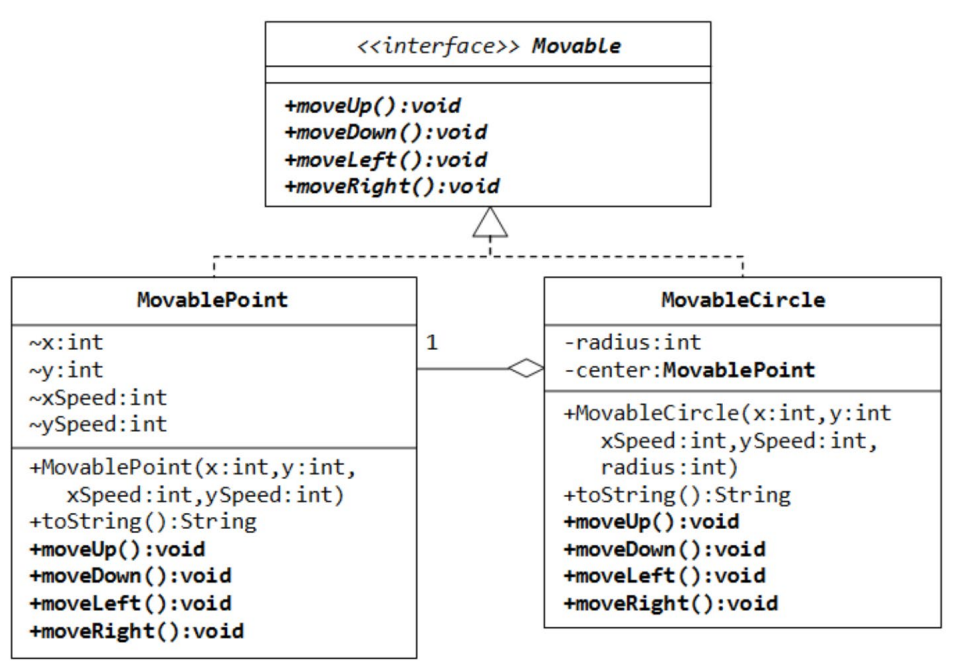


Source: https://www.dineshonjava.com/encapsulation-in-java/

Principle #3: Public Interface

OO CONCEPT

- For **objects** to work cooperatively and efficiently, clarify exactly how they should **interact, or interface**, with one another.
- Each object should present a **clear public interface** that determines how other objects will be used.



Source: https://stackoverflow.com/questions/28520164/why-are-interfaces-helpful

Principle #4: Information Hiding

OO CONCEPT

- To enable objects to work together cooperatively, **certain details** of their design and performance should be **hidden** from other objects.
- Each object should **shield** its users from **unnecessary details** of how it performs its role.

Hello.java | Comparator.class | Comparator.class | *Main.java | Hello.java | NewFile.xml

```
class Parent {

  public static void sleep() {
    System.out.println("Sleeps at 11 PM");
  }
}

class Child extends Parent {

  public static void sleep() {
    System.out.println("Sleeps at 9 PM");
  }
}

public class Code {

  public static void main(String[] args) {
    Parent p = new Parent();
    Parent c = new Child();
    p.sleep();
    c.sleep();

  }
}
```

Method Hiding

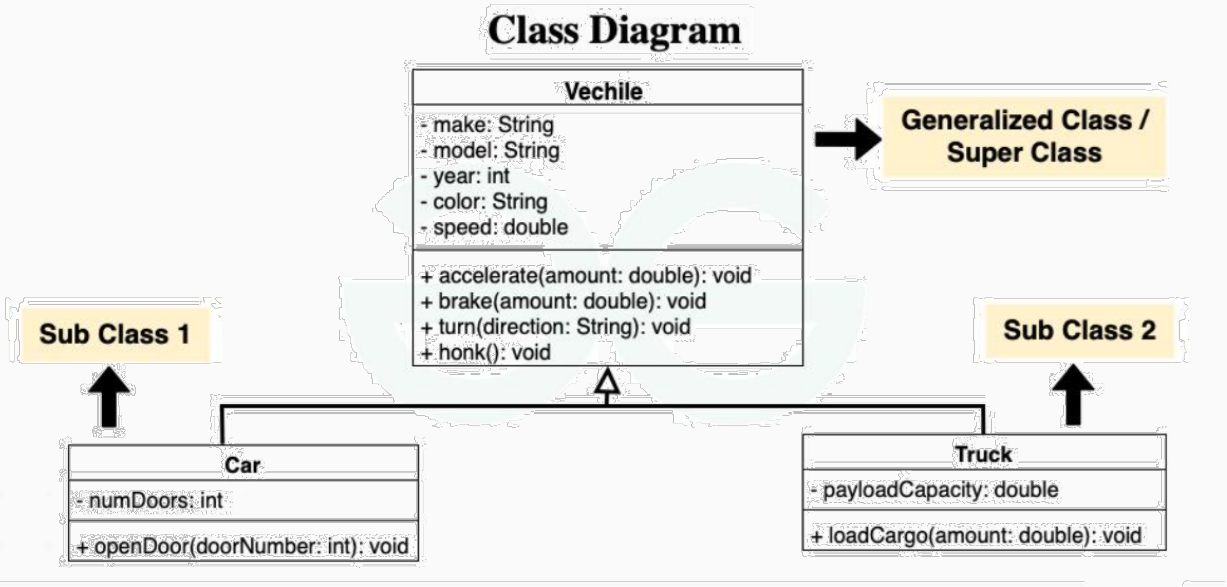
subclass mehod is hidden

Source: https://www.java67.com/2019/07/java-variable-and-method-hiding-example.html#google_vignette

Principle #5: Generality

OO CONCEPT

- To make an object as generally useful as possible, design them **not for a particular task but** for a **particular kind of task**.
- This principle underlies the use of software libraries.
- Objects should be designed to be as **general** as possible.
- Objects are designed to solve a kind of task rather than a singular task.

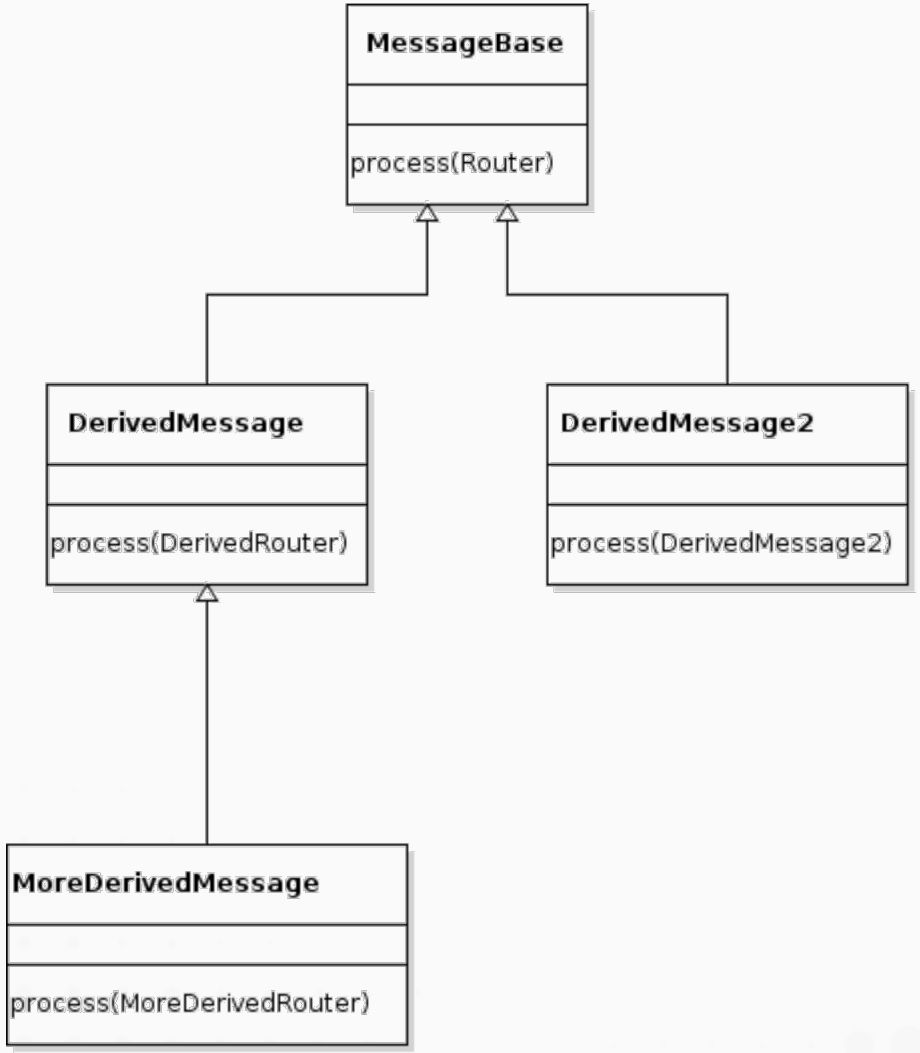


Source: https://www.geeksforgeeks.org/generalization-in-ooad/

Principle #6: Extensibility

OO CONCEPT

- One of the strengths of the OOP approach is the ability to extend an **object's behavior** to handle new tasks.
- An object should be designed so that its functionality **can be extended** to carry out more specialized tasks.

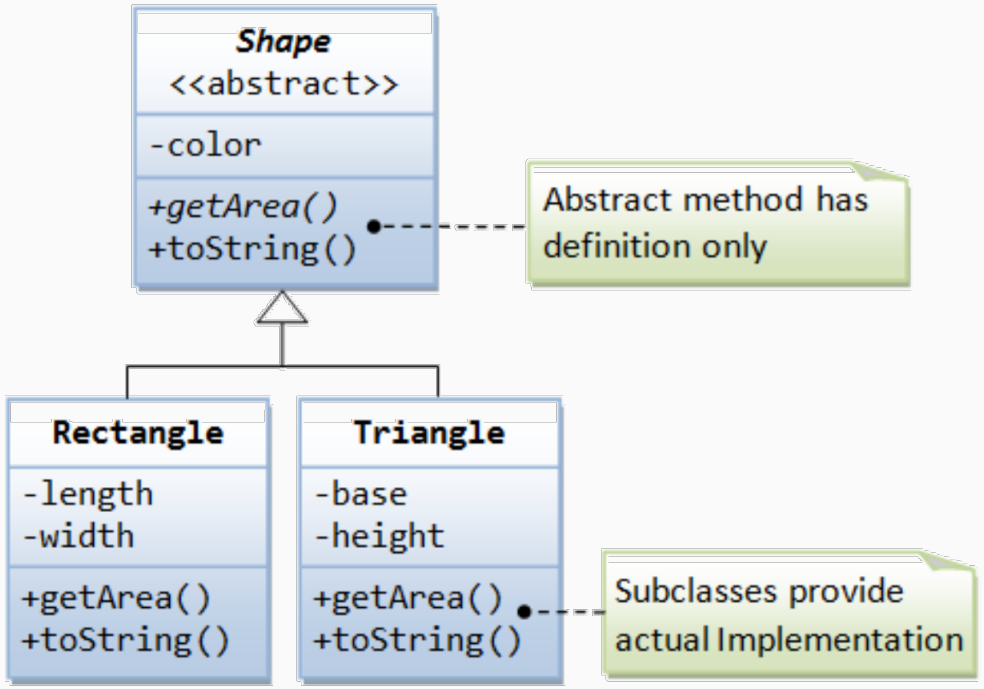


Source: https://stackoverflow.com/questions/28902349/two-way-extensible-hierarchy-with-java

Principle #7: Abstraction

OO CONCEPT

- Abstraction is the ability to **focus on the important features** of an object when trying to work with large amounts of information.
- The objects in Java programs will be abstractions in this sense because they **ignore many of the attributes** that characterize the real objects
- They **focus** only on those **attributes that are essential** for solving a particular problem.



Source:https://stackoverflow.com/questions/28038173/how-do-we-achieve-abstraction-using-interfaces-in-java

Benefits of OOP

- Save development time and cost by **reusing code**
  - Once an object class is created it can be used in other applications
- Easier **debugging**:
  - Classes can be tested independently
  - Reused objects have already been tested

Exercise

SELF-TEST

1. What are the **7 basic principles** of object orientation? Provide a brief description for each principle.
2. State the **benefits** of OOP.